DS HOMEWORK PROGRAMS EXERCISE 6 (2025115100)

1. Simulate Undo and Redo in MS Word using a Doubly Linked List

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node {
   char text[100];
   struct Node *prev;
   struct Node *next;
} Node;
Node *current = NULL;
void insertText(char text[]) {
   Node *newNode = (Node *)malloc(sizeof(Node));
   strcpy(newNode->text, text);
   newNode->prev = current;
   newNode->next = NULL;
   if (current != NULL) {
      Node *temp = current->next;
      while (temp != NULL) {
         Node *del = temp;
         temp = temp->next;
         free(del);
      }
      current->next = newNode;
   }

   newNode->prev = current;
   current = newNode;
}
void undo() {
   if (current != NULL && current->prev != NULL)
```

```
        current = current->prev;
    else
        printf("Nothing to undo\n");
}
void redo() {
    if (current != NULL && current->next != NULL)
        current = current->next;
    else
        printf("Nothing to redo\n");
}
void display() {
    if (current == NULL)
        printf("Document is empty\n");
    else
        printf("Document: %s\n", current->text);
}
int main() {
    int choice;
    char text[100];
    while (1) {
        printf("\n1.Insert Text\n2.Undo\n3.Redo\n4.Display\n5.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();
        switch (choice) {
            case 1:
                printf("Enter text: ");
                fgets(text, sizeof(text), stdin);
                text[strcspn(text, "\n")] = '\0';
                insertText(text);
                break;
```

```
            case 2:
                undo();
                break;
            case 3:
                redo();
                break;
            case 4:
                display();
                break;
            case 5:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}
```

OUTPUT

1.Insert Text

2.Undo

3.Redo

4.Display

5.Exit

Enter choice: 1

Enter text: Hello

Enter choice: 1

Enter text: Hello World

Enter choice: 4

Document: Hello World

Enter choice: 2

Enter choice: 4

Document: Hello

Enter choice: 3

Enter choice: 4

Document: Hello World

Enter choice: 5

2. Find the Intersection and Union of Two Sorted Lists

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
   int data;
   struct Node *next;
} Node;

void insert(Node **head, int value) {
   Node *newNode = malloc(sizeof(Node));
   newNode->data = value;
   newNode->next = NULL;

   if (*head == NULL) {
      *head = newNode;
      return;
   }

   Node *temp = *head;
   while (temp->next != NULL)
      temp = temp->next;
```

```
    temp->next = newNode;
}

void display(Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

Node *intersection(Node *a, Node *b) {
    Node *result = NULL;

    while (a != NULL && b != NULL) {
        if (a->data == b->data) {
            insert(&result, a->data);
            a = a->next;
            b = b->next;
        } else if (a->data < b->data) {
            a = a->next;
        } else {
            b = b->next;
        }
    }

    return result;
}

Node *unionList(Node *a, Node *b) {
    Node *result = NULL;
```

```
    while (a != NULL && b != NULL) {
        if (a->data < b->data) {
            insert(&result, a->data);
            a = a->next;
        } else if (a->data > b->data) {
            insert(&result, b->data);
            b = b->next;
        } else {
            insert(&result, a->data);
            a = a->next;
            b = b->next;
        }
    }

    while (a != NULL) {
        insert(&result, a->data);
        a = a->next;
    }

    while (b != NULL) {
        insert(&result, b->data);
        b = b->next;
    }

    return result;
}

int main() {
    Node *L1 = NULL, *L2 = NULL;
    Node *inter, *uni;
```

```
    int n, m, i, x;

    printf("Enter number of elements in L1: ");
    scanf("%d", &n);

    printf("Enter sorted elements of L1: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        insert(&L1, x);
    }
    printf("Enter number of elements in L2: ");
    scanf("%d", &m);
    printf("Enter sorted elements of L2: ");
    for (i = 0; i < m; i++) {
        scanf("%d", &x);
        insert(&L2, x);
    }
    inter = intersection(L1, L2);
    uni = unionList(L1, L2);
    printf("Intersection: ");
    display(inter)
    printf("Union: ");
    display(uni);

    return 0;
}
```

OUTPUT

```
Enter number of elements in L1: 4
Enter sorted elements of L1: 1 2 3 5
Enter number of elements in L2: 4
Enter sorted elements of L2: 2 3 4 5
```

Intersection: 2 3 5

Union: 1 2 3 4 5

3. Swap Two Elements by Adjusting Only Pointers in a Doubly Linked List

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
   int data;
   struct Node *prev;
   struct Node *next;
} Node;

Node *head = NULL;

void insert(int value) {
   Node *newNode = malloc(sizeof(Node));
   newNode->data = value;
   newNode->prev = NULL;
   newNode->next = NULL;

   if (head == NULL) {
      head = newNode;
      return;
   }

   Node *temp = head;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
   newNode->prev = temp;
}

void display() {
   Node *temp = head;
   while (temp != NULL) {
      printf("%d ", temp->data);
      temp = temp->next;
   }
   printf("\n");
}

void swapNodes(int x, int y) {
   Node *a = head, *b = head, *temp;

   while (a != NULL && a->data != x)
```

```
    a = a->next;

while (b != NULL && b->data != y)
    b = b->next;

if (a == NULL || b == NULL) {
    printf("Element not found\n");
    return;
}

if (a == b)
    return;

if (a->next == b) {
    temp = a;
    a = b;
    b = temp;
}

if (b->next == a) {
    temp = a;
    a = b;
    b = temp;
}

if (a->next == b) {
    Node *ap = a->prev;
    Node *bn = b->next;

    if (ap != NULL)
        ap->next = b;
    else
        head = b;

    if (bn != NULL)
        bn->prev = a;

    b->prev = ap;
    b->next = a;
    a->prev = b;
    a->next = bn;
    return;
}

Node *ap = a->prev;
Node *an = a->next;
Node *bp = b->prev;
Node *bn = b->next;
```

```
    if (ap != NULL)
        ap->next = b;
    else
        head = b;

    if (an != NULL)
        an->prev = b;

    if (bp != NULL)
        bp->next = a;
    else
        head = a;

    if (bn != NULL)
        bn->prev = a;

    a->prev = bp;
    a->next = bn;
    b->prev = ap;
    b->next = an;
}

int main() {
    int n, i, x, y, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        insert(value);
    }

    printf("Original list: ");
    display();

    printf("Enter two elements to swap: ");
    scanf("%d %d", &x, &y);

    swapNodes(x, y);

    printf("List after swapping: ");
    display();

    return 0;
}
```

OUTPUT
Enter number of nodes: 5
Enter elements: 10 20 30 40 50
Original list: 10 20 30 40 50
Enter two elements to swap: 20 40
List after swapping: 10 40 30 20 50

4. Array Implementation of a Self-Adjusting List

```
#include <stdio.h>

#define MAX 100

int list[MAX];
int size = 0;

void insert(int value) {
   int i;

   if (size == MAX) {
      printf("List is full\n");
      return;
   }

   for (i = size; i > 0; i--)
      list[i] = list[i - 1];

   list[0] = value;
   size++;
}

void find(int value) {
   int i, j, found = 0, temp;

   for (i = 0; i < size; i++) {
      if (list[i] == value) {
         found = 1;
         temp = list[i];

         for (j = i; j > 0; j--)
            list[j] = list[j - 1];

         list[0] = temp;
         printf("Element found\n");
         return;
      }
   }
```

```
    if (!found)
        printf("Element not found\n");
}

void display() {
    int i;

    printf("List: ");
    for (i = 0; i < size; i++)
        printf("%d ", list[i]);

    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert\n2.Find\n3.Display\n4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter element: ");
                scanf("%d", &value);
                insert(value);
                break;

            case 2:
                printf("Enter element to find: ");
                scanf("%d", &value);
                find(value);
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }
}
```

OUTPUT

1.Insert
2.Find
3.Display
4.Exit
Enter choice: 1
Enter element: 10

Enter choice: 1
Enter element: 20

Enter choice: 1
Enter element: 30

Enter choice: 3
List: 30 20 10

Enter choice: 2
Enter element to find: 10
Element found

Enter choice: 3
List: 10 30 20

Enter choice: 4

5. C program to reverse a given doubly linked list

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev, *next;
};

struct node *head = NULL;

void insert(int data) {
   struct node *newnode = malloc(sizeof(struct node));
   newnode->data = data;
   newnode->prev = NULL;
   newnode->next = head;
   if (head != NULL)
      head->prev = newnode;
   head = newnode;
}

void reverse() {
   struct node *temp = NULL, *current = head;
```

```
    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }

    if (temp != NULL)
        head = temp->prev;
}

void display() {
    struct node *temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    insert(30);
    insert(20);
    insert(10);

    printf("Original list: ");
    display();

    reverse();

    printf("Reversed list: ");
    display();

    return 0;
}
```

OUTPUT

Original list: 10 20 30

Reversed list: 30 20 10

6. C program to delete the middle element of a doubly linked list (DLL)

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

struct node *head = NULL;
```

```
void insert(int data) {
   struct node *newnode = malloc(sizeof(struct node));
   struct node *temp;

   newnode->data = data;
   newnode->prev = NULL;
   newnode->next = NULL;

   if (head == NULL) {
      head = newnode;
      return;
   }

   temp = head;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newnode;
   newnode->prev = temp;
}

void deleteMiddle() {
   struct node *slow = head, *fast = head;

   if (head == NULL)
      return;

   while (fast != NULL && fast->next != NULL) {
      slow = slow->next;
      fast = fast->next->next;
   }

   if (slow->prev != NULL)
      slow->prev->next = slow->next;
   else
      head = slow->next;

   if (slow->next != NULL)
      slow->next->prev = slow->prev;

   free(slow);
}

void display() {
   struct node *temp = head;
   while (temp != NULL) {
      printf("%d ", temp->data);
```

```
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    insert(10);
    insert(20);
    insert(30);
    insert(40);
    insert(50);

    printf("Original list: ");
    display();

    deleteMiddle();

    printf("After deleting middle: ");
    display();

    return 0;
}
```

OUTPUT

Original list: 10 20 30 40 50

After deleting middle: 10 20 40 50

7. (i) C program to remove duplicate elements from a sorted DLL

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

struct node *head = NULL;

void insert(int data) {
    struct node *newnode = malloc(sizeof(struct node));
    struct node *temp;

    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = NULL;

    if (head == NULL) {
        head = newnode;
        return;
    }
```

```
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newnode;
    newnode->prev = temp;
}

void removeDuplicates() {
    struct node *current = head, *temp;

    while (current != NULL && current->next != NULL) {
        if (current->data == current->next->data) {
            temp = current->next;
            current->next = temp->next;

            if (temp->next != NULL)
                temp->next->prev = current;

            free(temp);
        } else {
            current = current->next;
        }
    }
}

void display() {
    struct node *temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    insert(10);
    insert(10);
    insert(20);
    insert(20);
    insert(30);
    insert(30);

    printf("Original list: ");
    display();

    removeDuplicates();
```

```
    printf("After removing duplicates: ");
    display();

    return 0;
}
```

OUTPUT

Original list: 10 10 20 20 30 30

After removing duplicates: 10 20 30

ii. C program to remove duplicate elements from an unsorted DLL

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

struct node *head = NULL;

void insert(int data) {
    struct node *newnode = malloc(sizeof(struct node));
    struct node *temp;

    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = NULL;

    if (head == NULL) {
        head = newnode;
        return;
    }

    temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newnode;
    newnode->prev = temp;
}

void removeDuplicates() {
    struct node *current = head, *runner, *temp;

    while (current != NULL) {
        runner = current;

        while (runner->next != NULL) {
```

```
            if (runner->next->data == current->data) {
               temp = runner->next;
               runner->next = temp->next;

               if (temp->next != NULL)
                  temp->next->prev = runner;

               free(temp);
            } else {
               runner = runner->next;
            }
         }

         current = current->next;
      }
   }

   void display() {
      struct node *temp = head;
      while (temp != NULL) {
         printf("%d ", temp->data);
         temp = temp->next;
      }
      printf("\n");
   }

   int main() {
      insert(10);
      insert(20);
      insert(10);
      insert(30);
      insert(20);
      insert(40);

      printf("Original list: ");
      display();

      removeDuplicates();

      printf("After removing duplicates: ");
      display();

      return 0;
   }
```

Output

```
Original list: 10 20 10 30 20 40
After removing duplicates: 10 20 30 40
```

8. C program to calculate the frequency count of a given integer in a circular doubly linked list

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev, *next;
};

struct node *head = NULL;

void insert(int data) {
   struct node *newnode = malloc(sizeof(struct node));
   struct node *last;

   newnode->data = data;

   if (head == NULL) {
      head = newnode;
      newnode->next = head;
      newnode->prev = head;
      return;
   }

   last = head->prev;
   newnode->next = head;
   newnode->prev = last;
   last->next = newnode;
   head->prev = newnode;
}

int frequency(int key) {
   struct node *temp = head;
   int count = 0;

   if (head == NULL)
      return 0;

   do {
      if (temp->data == key)
         count++;
      temp = temp->next;
   } while (temp != head);

   return count;
}

int main() {
```

```
    int key;

    insert(10);
    insert(20);
    insert(10);
    insert(30);
    insert(10);

    printf("Enter the element to count: ");
    scanf("%d", &key);

    printf("Frequency of %d = %d\n", key, frequency(key));

    return 0;
}
```

Output

```
Enter the element to count: 10
Frequency of 10 = 3
```

9. C program to search for a particular element in a binary tree

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node *create(int data) {
    struct node *newnode = malloc(sizeof(struct node));
    newnode->data = data;
    newnode->left = NULL;
    newnode->right = NULL;
    return newnode;
}

int search(struct node *root, int key) {
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    return search(root->left, key) || search(root->right, key);
}

int main() {
    int key;
```

```
    struct node *root = create(10);
    root->left = create(20);
    root->right = create(30);
    root->left->left = create(40);
    root->left->right = create(50);

    printf("Enter the element to search: ");
    scanf("%d", &key);

    if (search(root, key))
        printf("Element found\n");
    else
        printf("Element not found\n");

    return 0;
}
```

Output

```
Enter the element to search: 50
Element found
```

10. C program to calculate the height or depth of a tree and print it

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node *create(int data) {
    struct node *newnode = malloc(sizeof(struct node));
    newnode->data = data;
    newnode->left = NULL;
    newnode->right = NULL;
    return newnode;
}

int height(struct node *root) {
    int leftHeight, rightHeight;

    if (root == NULL)
        return 0;

    leftHeight = height(root->left);
    rightHeight = height(root->right);

    if (leftHeight > rightHeight)
        return leftHeight + 1;
    else
```

```
        return rightHeight + 1;
    }

    int main() {
      struct node *root = create(10);
      root->left = create(20);
      root->right = create(30);
      root->left->left = create(40);
      root->left->right = create(50);

      printf("Height of the tree = %d\n", height(root));

      return 0;
    }
```

Output

Height of the tree = 3

11. Given a binary tree, convert this to a doubly linked list keeping same order (inorder traversal).

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

struct Node* create(int data) {
  struct Node *n = malloc(sizeof(struct Node));
  n->data = data;
  n->left = n->right = NULL;
  return n;
}

void convert(struct Node *root, struct Node **head,
        struct Node **prev) {
  if (root == NULL)
```

```
        return;

    convert(root->left, head, prev);

    if (*prev == NULL)
        *head = root;
    else {
        root->left = *prev;
        (*prev)->right = root;
    }

    *prev = root;

    convert(root->right, head, prev);
}

int main() {
    struct Node *root = create(10);
    root->left = create(5);
    root->right = create(20);
    root->left->left = create(2);
    root->left->right = create(8);
    root->right->left = create(15);
    root->right->right = create(25);

    struct Node *head = NULL;
    struct Node *prev = NULL;

    convert(root, &head, &prev);

    if (prev != NULL)
```

```
        prev->right = NULL;

    printf("Doubly Linked List (Forward):\n");
    struct Node *temp = head;

    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->right != NULL)
            printf(" <-> ");
        temp = temp->right;
    }

    printf("\nDoubly Linked List (Backward):\n");
    temp = prev;

    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->left != NULL)
            printf(" <-> ");
        temp = temp->left;
    }

    printf("\n");
    return 0;
}
```

Output

```
Doubly Linked List (Forward):
2 <-> 5 <-> 8 <-> 10 <-> 15 <-> 20 <-> 25
Doubly Linked List (Backward):
25 <-> 20 <-> 15 <-> 10 <-> 8 <-> 5 <-> 2
```

12. Connect nodes at the same level using next-right pointers

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right, *next;
};

struct Node* create(int data) {
   struct Node *n = malloc(sizeof(struct Node));
   n->data = data;
   n->left = n->right = n->next = NULL;
   return n;
}

void connect(struct Node *root) {
   if (root == NULL)
      return;

   struct Node *queue[100];
   int front = 0, rear = 0;
   queue[rear++] = root;

   while (front < rear) {
      int count = rear - front;
      struct Node *prev = NULL;

      for (int i = 0; i < count; i++) {
         struct Node *cur = queue[front++];

         if (prev != NULL)
```

```
            prev->next = cur;

        prev = cur;

        if (cur->left)
            queue[rear++] = cur->left;

        if (cur->right)
            queue[rear++] = cur->right;
      }

      prev->next = NULL;
   }
}

void display(struct Node *root) {
   struct Node *level = root;

   while (level != NULL) {
      struct Node *cur = level;

      while (cur != NULL) {
         printf("%d -> ", cur->data);
         cur = cur->next;
      }

      printf("NULL\n");

      if (level->left)
         level = level->left;
      else if (level->right)
```

```
            level = level->right;
        else
            break;
    }
}

int main() {
    struct Node *root = create(1);
    root->left = create(2);
    root->right = create(3);
    root->left->left = create(4);
    root->left->right = create(5);
    root->right->right = create(6);

    connect(root);

    printf("Nodes connected level-wise:\n");
    display(root);

    return 0;
}
```

Output

```
Nodes connected level-wise:
1 -> NULL
2 -> 3 -> NULL
4    -> 5 -> 6 -> NULL
```

13.Implement a priority queue using a binary search tree — insertion

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int priority;
    struct Node *left, *right;
```

```
};

struct Node* insert(struct Node *root, int p) {
   if (root == NULL) {
      struct Node *n = malloc(sizeof(struct Node));
      n->priority = p;
      n->left = n->right = NULL;
      return n;
   }

   if (p < root->priority)
      root->left = insert(root->left, p);
   else if (p > root->priority)
      root->right = insert(root->right, p);

   return root;
}

void display(struct Node *root) {
   if (root != NULL) {
      display(root->right);
      printf("%d ", root->priority);
      display(root->left);
   }
}

int main() {
   struct Node *root = NULL;
   int n, p;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   for (int i = 0; i < n; i++) {
      printf("Enter priority: ");
      scanf("%d", &p);
      root = insert(root, p);
   }

   printf("Priority queue (highest first): ");
   display(root);

   return 0;
}
```

Output

```
Enter number of elements: 5
Enter priority: 30
Enter priority: 10
```

```
Enter priority: 50
Enter priority: 20
Enter priority: 40
Priority queue (highest first): 50 40 30 20 10
```

14.Delete a node in a binary search tree

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* create(int data) {
   struct Node *n = malloc(sizeof(struct Node));
   n->data = data;
   n->left = n->right = NULL;
   return n;
}

struct Node* insert(struct Node *root, int data) {
   if (root == NULL)
      return create(data);

   if (data < root->data)
      root->left = insert(root->left, data);
   else if (data > root->data)
      root->right = insert(root->right, data);

   return root;
}

struct Node* minimum(struct Node *root) {
   while (root->left != NULL)
      root = root->left;
   return root;
}

struct Node* deleteNode(struct Node *root, int key) {
   if (root == NULL)
      return NULL;

   if (key < root->data)
      root->left = deleteNode(root->left, key);
   else if (key > root->data)
      root->right = deleteNode(root->right, key);
   else {
```

```
        if (root->left == NULL) {
            struct Node *temp = root->right;
            free(root);
            return temp;
        }

        if (root->right == NULL) {
            struct Node *temp = root->left;
            free(root);
            return temp;
        }

        struct Node *temp = minimum(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct Node *root = NULL;
    int a[] = {50, 30, 70, 20, 40, 60, 80};
    int n = 7;

    for (int i = 0; i < n; i++)
        root = insert(root, a[i]);

    printf("Original tree: ");
    inorder(root);

    root = deleteNode(root, 20);
    printf("\nAfter deleting leaf node 20: ");
    inorder(root);

    root = deleteNode(root, 30);
    printf("\nAfter deleting node 30 with one child: ");
    inorder(root);

    root = deleteNode(root, 50);
```

```
    printf("\nAfter deleting node 50 with two children: ");
    inorder(root);

    return 0;
}
```

Output

```
Original tree: 20 30 40 50 60 70 80
After deleting leaf node 20: 30 40 50 60 70 80
After deleting node 30 with one child: 40 50 60 70 80
After deleting node 50 with two children: 40 60 70 80
```

15.Convert a postfix expression into an expression tree

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node* stack[100];
int top = -1;

struct Node* create(char data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->left = n->right = NULL;
    return n;
}

int isOperator(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

struct Node* buildTree(char exp[]) {
    for (int i = 0; exp[i] != '\0'; i++) {
        char c = exp[i];

        if (isalnum(c)) {
            stack[++top] = create(c);
        } else if (isOperator(c)) {
            struct Node *n = create(c);
            n->right = stack[top--];
            n->left = stack[top--];
            stack[++top] = n;
        }
    }
```

```
    return stack[top];
}

void preorder(struct Node *root) {
    if (root != NULL) {
        printf("%c ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void inorder(struct Node *root) {
    if (root != NULL) {
        if (isOperator(root->data))
            printf("(");
        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);
        if (isOperator(root->data))
            printf(")");
    }
}

int main() {
    char exp[] = "ab+c*";
    struct Node *root = buildTree(exp);

    printf("Postfix expression: %s\n", exp);
    printf("Prefix expression: ");
    preorder(root);
    printf("\nInfix expression: ");
    inorder(root);

    return 0;
}
```

Output

Postfix expression: ab+c*

Prefix expression: * + a b c

Infix expression: ((a+b)*c)

16.Construct a binary search tree and display inorder, preorder and postorder

```
#include <stdio.h>

#include <stdlib.h>


struct Node {
```

```
    int data;
    struct Node *left, *right;
};

struct Node* create(int data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->left = n->right = NULL;
    return n;
}

struct Node* insert(struct Node *root, int data) {
    if (root == NULL)
        return create(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```
void preorder(struct Node *root) {
   if (root != NULL) {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void postorder(struct Node *root) {
   if (root != NULL) {
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
   }
}

int main() {
   struct Node *root = NULL;
   int a[] = {50, 30, 70, 20, 40, 60, 80};
   int n = 7;

   for (int i = 0; i < n; i++)
      root = insert(root, a[i]);

   printf("Inorder: ");
   inorder(root);

   printf("\nPreorder: ");
   preorder(root);

   printf("\nPostorder: ");
```

```
    postorder(root);

    return 0;
}
```

Output

```
Inorder: 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50
```

17. Implement heap sort for a set of numbers

```
#include <stdio.h>

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && a[left] > a[largest])
        largest = left;

    if (right < n && a[right] > a[largest])
        largest = right;

    if (largest != i) {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}
```

```
void heapSort(int a[], int n) {
   int temp;

   for (int i = n / 2 - 1; i >= 0; i--)
      heapify(a, n, i);

   for (int i = n - 1; i > 0; i--) {
      temp = a[0];
      a[0] = a[i];
      a[i] = temp;

      heapify(a, i, 0);
   }
}

int main() {
   int a[] = {50, 20, 30, 15, 5, 22, 32};
   int n = sizeof(a) / sizeof(a[0]);

   printf("Original array: ");
   for (int i = 0; i < n; i++)
      printf("%d ", a[i]);

   heapSort(a, n);

   printf("\nSorted array: ");
   for (int i = 0; i < n; i++)
      printf("%d ", a[i]);

   return 0;
```

```
}
```

Output

```
Original array: 50 20 30 15 5 22 32
Sorted array: 5 15 20 22 30 32 50
```